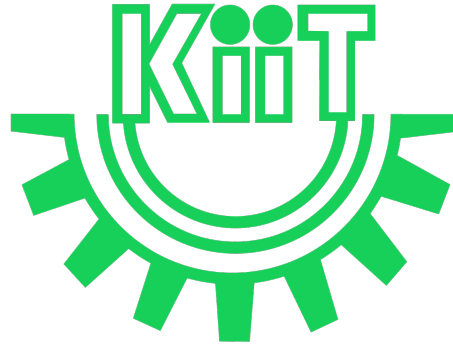




TCP Congestion Control Using NS2

Smruti Prangya (2305414)

Sneha Ghorai (2305415)

Thanusree Y. (2305425)

Ushasi Bhunia (23052445)

Kalinga Institute of Industrial Technology

Project Guide: Dr. Subhra Priyadarshini Biswal

13th April 2026

1 Abstract

The Transmission Control Protocol (TCP) is essentially a computer network protocol that ensures the reliability of transmitted data [1]. It carries most of the Internet traffic, and therefore the overall performance of the Internet largely depends on how efficiently TCP operates. TCP's congestion control algorithms play a significant role in maintaining network performance and stability. Network congestion has become a major issue in modern network analysis. This project focuses on the implementation and performance analysis of the CERL Congestion Estimation using Rate Limiting algorithm using Network Simulator 2 (NS2). TCP CERL is a sender-side modification of the TCP Reno protocol, designed to improve throughput by distinguishing between random packet loss and congestion-induced losses. In addition to CERL, other widely used TCP variants such as TCP Reno, TCP CUBIC, and TCP BBR are also considered for comparative analysis [2–4]. The algorithms are evaluated over two different network topologies, namely star and bus, to study their behavior under varying network structures. Simulations are conducted to measure key performance metrics such as throughput, packet loss, delay, and congestion window behavior. The results demonstrate that CERL provides smoother congestion control and reduced packet loss, while the comparative study highlights the performance differences among various TCP variants under different topological conditions.

2 Introduction

A computer network is a system of interconnected computing devices such as computers, servers, and mobile devices that exchange data and share resources like files and printers using wired or wireless connections. It enables efficient communication and supports services such as email, file sharing, and internet access. With the rapid growth of the Internet and network-based applications, efficient data transmission has become increasingly important. Network congestion occurs when the volume of data traffic exceeds the available network capacity, leading to degraded performance, increased delays, and packet loss. Transmission Control Protocol (TCP) plays a crucial role in managing data transmission and ensuring reliability [1]. TCP controls congestion by regulating the amount of data in transit (congestion window) based on network feedback. Traditional TCP congestion control mechanisms, such as TCP Reno, primarily rely on packet loss as an indicator of congestion [2]. To overcome these limitations, several advanced TCP variants have been developed, including TCP CUBIC and TCP BBR, which aim to improve throughput and network utilization [3, 4]. In addition, algorithms like CERL have been proposed to enhance performance by distinguishing between random packet loss and actual congestion [5]. In this project, the CERL algorithm is implemented and analyzed using the NS2 simulator, and its performance is compared with TCP Reno, TCP CUBIC, and TCP BBR. The evaluation is carried out over two different network topologies—star and bus—to understand the impact of network structure on congestion behavior.

2.1 Objective

- To implement and analyze the CERL algorithm using NS2.
- To compare CERL with TCP Reno, TCP CUBIC, and TCP BBR.
- To evaluate performance using throughput, delay, and packet loss.
- To study the impact of star and bus topologies on network performance.

3 Literature Review

This section presents a brief overview of the congestion control algorithms used for analysis.

3.1 TCP RENO

TCP Reno is a loss-based congestion control algorithm [2]. It increases the congestion window exponentially during slow start and linearly during congestion avoidance. Upon detecting congestion using duplicate acknowledgments, Reno retransmits the lost packet.

3.2 TCP CUBIC

TCP CUBIC uses a cubic function to control the congestion window [3]:

$$cwnd = C(t - K)^3 + W_{max} \quad (1)$$

3.3 TCP BBR

TCP BBR is a model-based algorithm that estimates bandwidth and round-trip time [4]:

$$Rate = Bandwidth \times RTT \quad (2)$$

3.4 CERL

CERL improves congestion control by distinguishing between random and congestion-related losses [5]. It uses delay-based estimation and rate limiting for better performance.

3.5 Performance Metrics

3.5.1 Throughput

$$Throughput = \frac{\text{Total data received}}{\text{Time}} \quad (3)$$

3.5.2 Packet Loss

$$Packet\ Loss = \frac{\text{Packets sent} - \text{Packets received}}{\text{Packets sent}} \quad (4)$$

3.5.3 End-To-End Delay

$$Delay = \text{Arrival time} - \text{Departure time} \quad (5)$$

3.5.4 Average Delay

$$Average\ Delay = \frac{\text{Total delay}}{\text{Number of packets}} \quad (6)$$

4 Methodology

The implementation is carried out using NS2 [6]. TCL scripts are used to define nodes, links, and traffic. This section describes the simulation setup, network topology design, traffic configuration, and implementation process used in the study.

4.1 Simulation Environment

The simulations are performed using NS2, which is widely used for modeling network protocols and analyzing their behavior. The simulation is executed using TCL scripts, and trace files are generated to record packet-level events such as packet transmission, reception, and packet drops.

4.2 Network Topology Design

Two different network topologies are used in this study to evaluate the behavior of congestion control algorithms under varying network structures.

4.2.1 Star Topology

In the star topology, all nodes are connected to a central node that acts as a hub for communication. Data transmission between nodes takes place through this central node, which may act as a bottleneck under heavy traffic conditions.

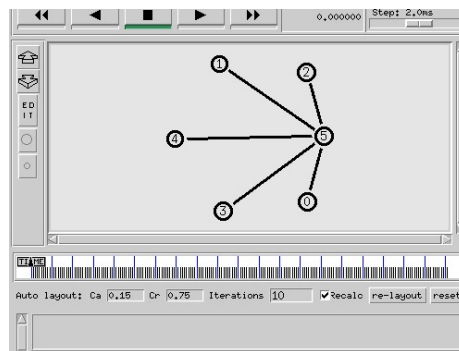


Figure 1: Star Topology

4.2.2 Bus Topology

In the bus topology, all nodes are connected to a common shared communication link. Data transmitted by any node is accessible to all other nodes connected to the network, making it suitable for analyzing shared medium communication.

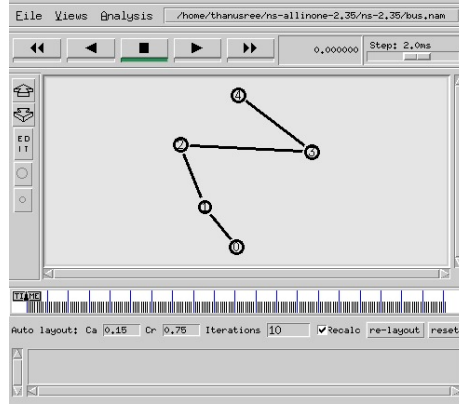


Figure 2: Bus Topology

4.3 Simulation Parameters

The network is configured using standard simulation parameters, including:

Parameter	Value
Link Bandwidth	1 Mb(Megabit per second)
Propagation Delay	10 ms(millisecond)
Queue Type	DropTail
Packet Size	1000 bytes
Simulation Time	20 seconds

Table 1: Simulation Parameters

These parameters are kept consistent for all simulations to ensure a fair comparison between different algorithms.

4.4 Traffic Configuration

Traffic is generated between selected source and destination nodes using TCP connections. Applications such as FTP are used to simulate continuous data transfer. TCP agents are attached to the nodes, and data transmission is carried out for a fixed duration during the simulation.

4.5 Implementation of Algorithms

The simulation includes the implementation of four congestion control algorithms:

TCP Reno
TCP CUBIC
TCP BBR
CERL

Each algorithm is implemented separately under identical network conditions for both star and bus topologies. This ensures consistency and allows for a proper comparison of their behavior.

4.6 Data Collection

During the simulation, trace files are generated that record all network events, including packet transmission, reception, and loss. These trace files are used for further processing to extract relevant performance data.

5 Result and Analysis

The simulation results obtained from NS2 are presented in this section. The performance of TCP Reno, TCP CUBIC, TCP BBR, and CERL is evaluated under star and bus topologies using different performance metrics.

5.1 Result For Star Topology

5.2 Congestion Window vs Time

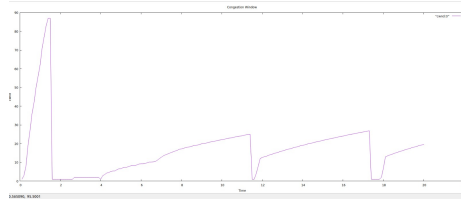


Figure 3: Congestion Window vs Time

5.2.1 Throughput Analysis

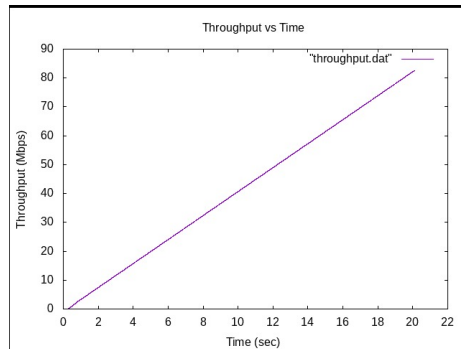


Figure 4: Throughput vs Time

5.2.2 Packet Loss Analysis

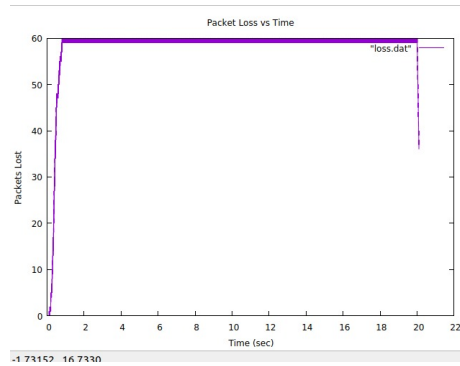


Figure 5: Packet Loss vs Time

5.2.3 Delay Analysis

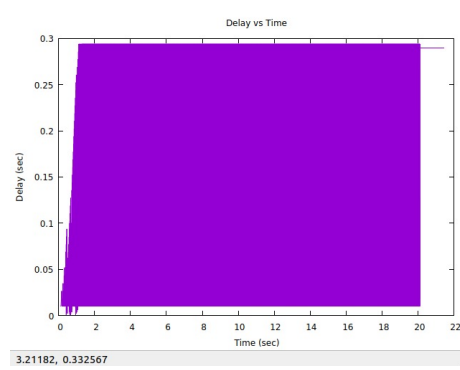


Figure 6: Delay vs Time

5.3 Result For Bus Topology

5.3.1 Throughput Analysis

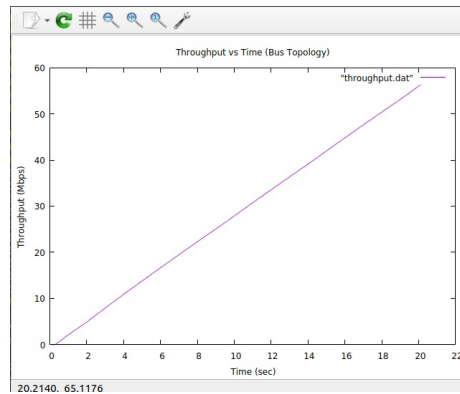


Figure 7: Throughput vs Time

5.3.2 Packet Loss Analysis

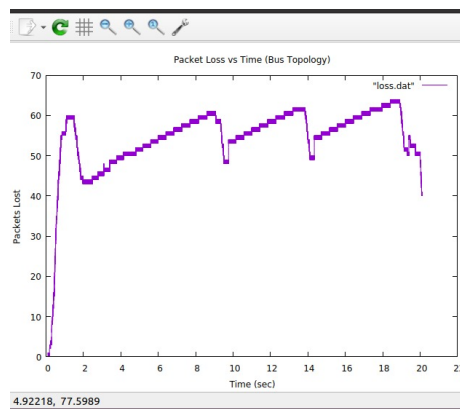


Figure 8: Packet Loss vs Time

5.3.3 Delay Analysis

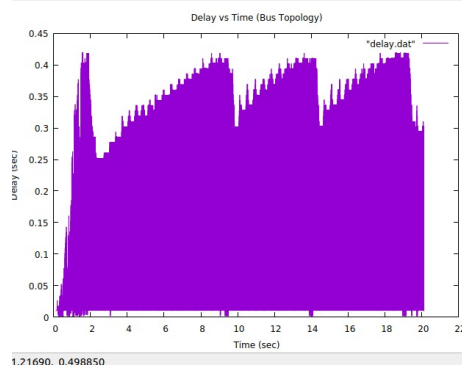


Figure 9: Delay vs Time

5.4 Comparative Table

5.5 Comparative Analysis

Table 2: Performance Metrics for Star Topology

Algorithm	Throughput	Packet Sent	Packet Received	Packet Loss	Delay
TCP Reno	2.216	10204	10158	46	0.07843
TCP CUBIC	3.7875	17567	17519	48	0.083
TCP BBR	4.6055	19716	19716	0	0.02092
CERL	4.1163	19133	18931	202	0.07676

Table 3: Performance Metrics for Bus Topology

Algorithm	Throughput	Packet Sent	Packet Received	Packet Loss	Delay
TCP Reno	2.8120	13018	12977	41	0.07008
TCP CUBIC	3.0984	14382	14318	64	0.0762
TCP BBR	2.524	12119	12116	3	0.0262
CERL	2.8449	13174	13131	43	0.07080

5.6 Discussion

The simulation results compare congestion control algorithms across star and bus topologies.

TCP BBR achieves the highest throughput due to efficient bandwidth estimation, followed by TCP CUBIC with fast window growth. CERL provides stable and high throughput, while TCP Reno performs lower due to its conservative approach.

In terms of packet loss, CERL performs best by distinguishing between random and congestion losses. TCP BBR also shows low loss, whereas Reno and CUBIC experience higher losses under congestion.

For delay, TCP BBR and CERL have lower end-to-end delay, while Reno shows higher delay and CUBIC remains moderate.

The congestion window behavior of CERL is smoother compared to the abrupt changes in Reno. CUBIC grows faster, while BBR maintains a stable rate.

Regarding topology, star performs better due to centralized control, while bus suffers from higher congestion, delay, and packet loss.

Overall, CERL offers better stability, and TCP BBR delivers the highest throughput, showing that both algorithm choice and topology impact network performance.

6 References

References

- [1] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*.
- [2] W. Stevens, *TCP/IP Illustrated*.
- [3] S. Ha et al., "CUBIC: A New TCP-Friendly High-Speed TCP Variant," 2008.
- [4] N. Cardwell et al., "BBR: Congestion-Based Congestion Control," 2016.
- [5] Research papers on CERL algorithm.
- [6] NS2 Documentation, <http://www.isi.edu/nsnam/ns/>